Faculty of Engineering – Zagazig University
Computer and Systems Engineering

CSE 226: Hardware Design

16-bit Cryptographic Coprocessor

Project Report

Prepared by:

No.	Name	ID	Work	Gmail
1	Omar Mohamed Ghamry Amin	20812023100477	Co-processor Integration	omarghamry2005@gmail.com
2	Farah Ahmed Hamza Osman	20812023200330	LUT	farahhamzawy286@gmail.com
3	Omar Mohamed Abdelrazek Elgrabawy	20812023100034	ALU	omarabdelrazek13@gmail.com
4	Farah Fayez Abdo Gerges	20812023100696	Register File	farahfayezabdou@gmail.com
5	Mohamed Ahmed Mohamed Zakaria	20812023100008	Shifter	mohamedzakaria532005@gmail.com

Supervisor

Dr. Howida Abd AlLatif

Academic Year

2025/2026

Contents

Assignment Aim

The aim of this project is to design and implement a simple Co-Processor using VHDL. The system performs a set of arithmetic and logical operations based on control signals, simulating the fundamental functionality of a processor unit.

This project focuses on developing a structured hardware design that includes input registers, control logic, and a processing unit. It also aims to enhance understanding of digital system design, hardware description using VHDL, and the interaction between datapath and control units.

The Co-Processor designed in this project supports the following operations:

- **Arithmetic:** 16-bit addition and two's-complement subtraction.
- **Bitwise Logic:** AND, OR, XOR, and NOT operations.
- **Bit Manipulation:** Right rotations (8-bit and 4-bit) and an 8-bit logical left shift.
- **Non-linear Substitution:** An S-Box lookup table (LUT) transformation that applies a non-linear mapping to an 8-bit input.

The design is structured **hierarchically**, starting from a generic *N-bit ripple-carry adder* up to a top-level `co_processor` entity.

Problem Solution

2.1 Workflow

The workflow of the Co-Processor describes the sequence of operations performed from input acquisition to result generation. The system operates synchronously with the clock signal and follows a structured datapath controlled by the control unit.

The workflow can be summarized as follows:

(i) **Initialization:** When the reset signal (`rst`) is activated, all internal registers and outputs are initialized to a known state.

(ii) **Input Loading:** The input operands (`Ra` and `Rb`) and control signal (`ctrl`) are provided to the Co-Processor.

(iii) **Operation Selection:** The control unit decodes the `ctrl` signal to determine the required operation (arithmetic, logical, bit manipulation, or S-Box transformation).

(iv) **Execution:** Based on the selected operation:

- Arithmetic operations are performed using the ripple-carry adder.
- Logical operations are executed using bitwise logic circuits.
- Bit manipulation operations perform shifting or rotation.
- The S-Box applies a non-linear transformation using a lookup table.

(v) **Result Generation:** The computed result is stored in the destination register (`Rd`).

(vi) **Output Update:** The output is updated on the next clock cycle, ensuring synchronous operation.

2.2 Inputs and Outputs

The Co-Processor is designed to perform various operations based on control signals and input data. The system interacts with external components through a set of defined inputs and outputs.

Inputs

- **clk(1-bit):** Clock signal used to synchronize operations.
- **rst(1-bit):** Reset signal used to initialize the system.
- **ctrl(4-bit):** Control signal that selects the required operation.
- **Ra(4-bit) & Rb(4-bit):** First & Second input operand.
- **Rd(4-bit):** Address of the destination register (result write-back).

2.2.1 Control Word Encoding

the table 2.1 below shows the operation in the Co-Processor controlled by ctrl

Table 2.1: CTRL encoding and corresponding operations

ctrl	instruction	operation
0000	ADD	unsigned addition Rd <= Ra + Rb
0001	SUB	(2's complement) Rd <= Ra - Rb
0010	AND	Bitwise AND: Rd <= Ra AND Rb
0011	OR	Bitwise OR: Rd <= Ra OR Rb
0100	XOR	Bitwise XOR: Rd <= Ra XOR Rb
0101	NOT	Bitwise NOT: Rd <= Ra
0110	MOV	allow Ra to pass Rd <= Ra
1000	ROR8	8-bit right rotation (7 downto 0) & (15 downto 8)
1001	ROR4	4-bit right rotation (3 downto 0) & (15 downto 4)
1010	SLL8	8-bit left shift (7 downto 0) & "00000000"
0111	NOP	DO not allow to write to the Register file
1011	LUT	RESULT <= OUTPUT of LOOKUP TABLE Implementation
others	—	defaults to ADD behaviour in the ALU path

Implementation

3.1 The nonlinear lookup operation unit

3.1.1 description

The nonlinear operation which is used in hashing algorithm utilizes a parallel 4-bit nonlinear operation where the input nibble (4 bits) are mapped to another nonlinear 4-bit value.

The details of the lookup table implementation are as follows:

S-Box 1		S-Box 2	
Input	Output	input	Output
"0000"	"0001"	"0000"	"1111"
"0001"	"1011"	"0001"	"0000"
"0010"	"1001"	"0010"	"1101"
"0011"	"1100"	"0011"	"0111"
"0100"	"1101"	"0100"	"1011"
"0101"	"0110"	"0101"	"1110"
"0110"	"1111"	"0110"	"0101"
"0111"	"0011"	"0111"	"1010"
"1000"	"1110"	"1000"	"1001"
"1001"	"1000"	"1001"	"0010"
"1010"	"0111"	"1010"	"1100"
"1011"	"0100"	"1011"	"0001"
"1100"	"1010"	"1100"	"0011"
"1101"	"0010"	"1101"	"0100"
"1110"	"0101"	"1110"	"1000"
"1111"	"0000"	"1111"	"0110"

Figure 3.1: details of lookup operation unit

3.1.2 Schematic diagram

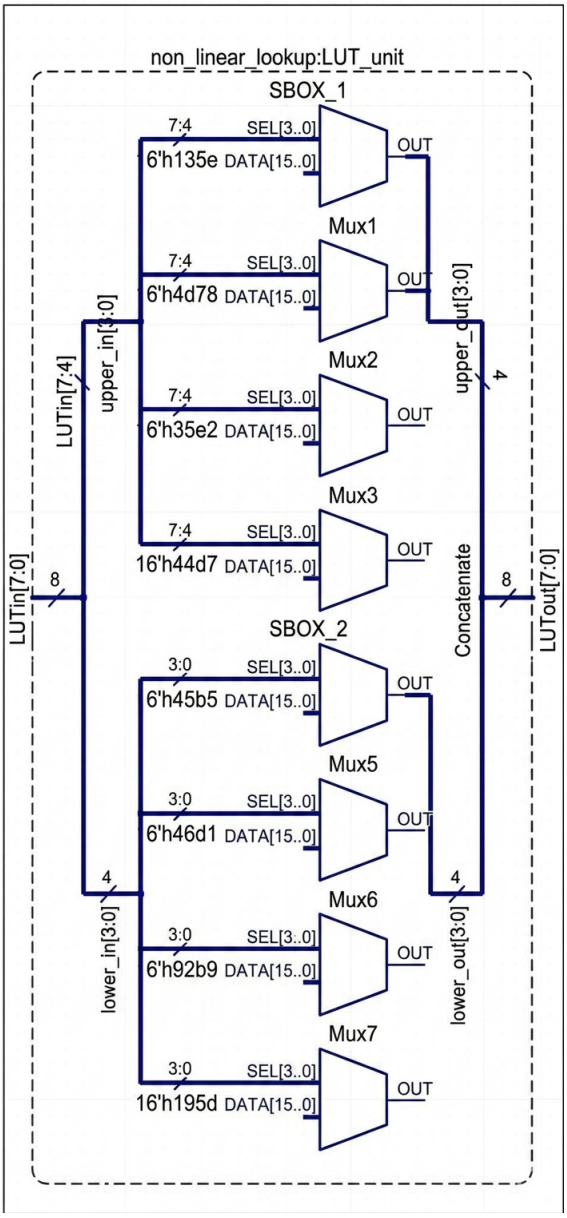

Figure 3.2: details of lookup operation unit

3.1.3 VHDL Code

VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity nonlinear_lut is
port (
LUTin : in std_logic_vector(7 downto 0);
```

```
LUTout : out std_logic_vector(7 downto 0) );
end entity ;
architecture rtl of nonlinear_lut is

signal upper_in, lower_in, upper_out, lower_out : std_logic_vector(3
downto 0);

begin
upper_in <= LUTin(7 downto 4);
lower_in <= LUTin(3 downto 0);
SBOX_1 : process (upper_in)
begin
case(upper_in) is
when "0000" => upper_out <= "0001";
when "0001" => upper_out <= "1011";
when "0010" => upper_out <= "1001";
when "0011" => upper_out <= "1100";
when "0100" => upper_out <= "1101";
when "0101" => upper_out <= "0110";
when "0110" => upper_out <= "1111";
when "0111" => upper_out <= "0011";
when "1000" => upper_out <= "1110";
when "1001" => upper_out <= "1000";
when "1010" => upper_out <= "0111";
when "1011" => upper_out <= "0100";
when "1100" => upper_out <= "1010";
when "1101" => upper_out <= "0010";
when "1110" => upper_out <= "0101";
when "1111" => upper_out <= "0000";
when others => upper_out <= "0000";
end case;
end process;
```

```
SBOX_2 : process (lower_in)
begin
case(lower_in) is
when "0000" => lower_out <= "1111";
when "0001" => lower_out <= "0000";
when "0010" => lower_out <= "1101";
when "0011" => lower_out <= "0111";
when "0100" => lower_out <= "1011";
when "0101" => lower_out <= "1110";
when "0110" => lower_out <= "0101";
when "0111" => lower_out <= "1010";
when "1000" => lower_out <= "1001";
when "1001" => lower_out <= "0010";
when "1010" => lower_out <= "1100";
when "1011" => lower_out <= "0001";
when "1100" => lower_out <= "0011";
when "1101" => lower_out <= "0100";
when "1110" => lower_out <= "1000";
when "1111" => lower_out <= "0110";
when others => lower_out <= "0000";
end case;
end process;
LUTout <= upper_out & lower_out;
end architecture;
```

3.1.4 Simulation of LUT

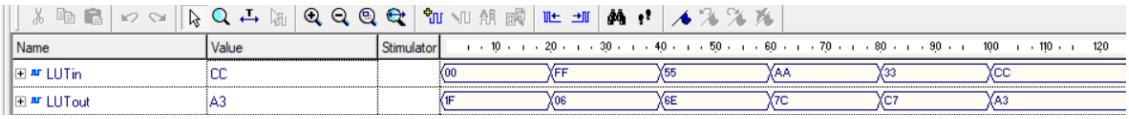

Figure 3.3: Simulation waveform for the nonlinear lookup table in VHDL

3.2 Shifter

3.2.1 description

shifter with the ability to shift and rotate data, which is mainly used in the permutation and transpositions of ciphers, will be implemented in VHDL. The VHDL shifter is a key component in the upcoming co-processor's processing unit. Fast shifting and rotating functions are critical for cryptographic applications.

The architecture apply different shift operations:

ctrl	instruction	Operation
1000	ROR8	performs an 8-bit right rotation
1001	ROR4	performs an 4-bit right rotation
1010	SLL8	performs an 8-bit left shift

3.2.2 Schematic diagram

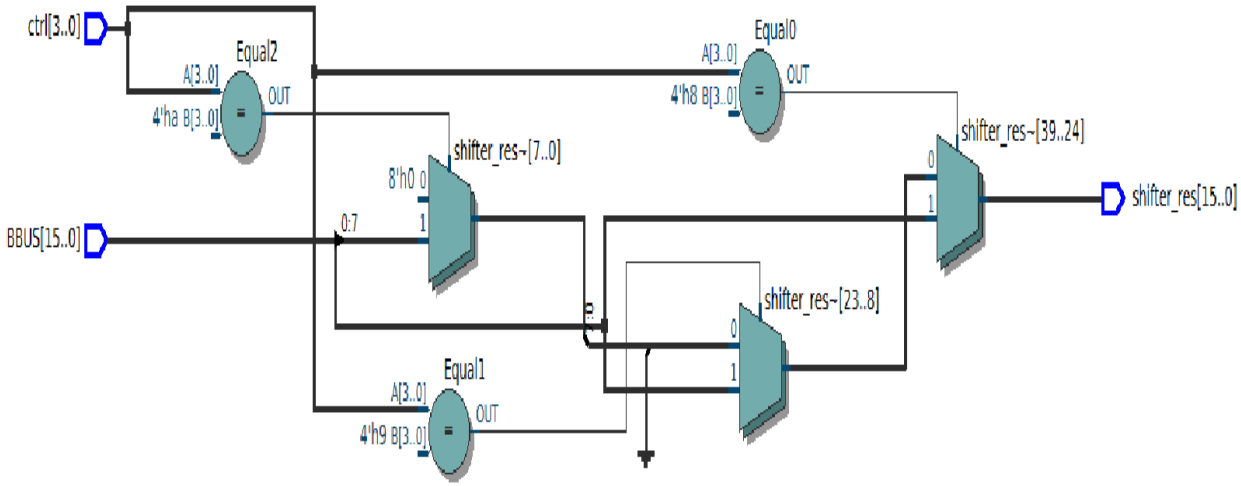

Figure 3.4: Schematic diagram of Shifter

3.2.3 VHDL Code

VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
entity shifter is
port (
ctrl:in std_logic_vector(3 downto 0);
BBUS:in std_logic_vector(15 downto 0);
shifter_res:out std_logic_vector(15 downto 0)
);
end entity;

architecture sft of shifter is
begin
process (ctrl,BBUS)
begin

if ctrl =”1000” then
shifter_res<= BBUS(7 downto 0) & BBUS(15 downto 8); – (ROR8).

elsif ctrl =”1001” then
shifter_res<= BBUS(3 downto 0) & BBUS(15 downto 4); –(ROR4).

elsif ctrl =”1010” then
shifter_res<= BBUS(7 downto 0) & x”00”; – (SLL8), filling with zeros

else
shifter_res<= (others =>’0’);
end if;



end process;
end architecture;
```

3.2.4 Simulation of Shifter

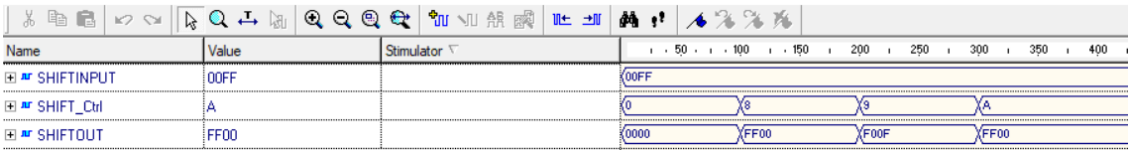

Figure 3.5: Simulation of Shifter

3.3 ALU

3.3.1 description

arithmetic logic unit (ALU) is a combinational digital circuit that performs arithmetic (ADD,Sub) and bitwise operations (AND,OR,XOR,NOT) .

inputs : ABUS (16-bit) , BBUS (16-bit) , Ctrl (4-bit)

output : ALU_res (16-bit) <=input for mux and selected by ctrl

For Addition/Subtraction We Used adder/subtractor combinational circuit to implement unsigned addiation and 2's complement subtraction
(as shown in figure 3.6 below)

3.3.1.1 adder-subtractor

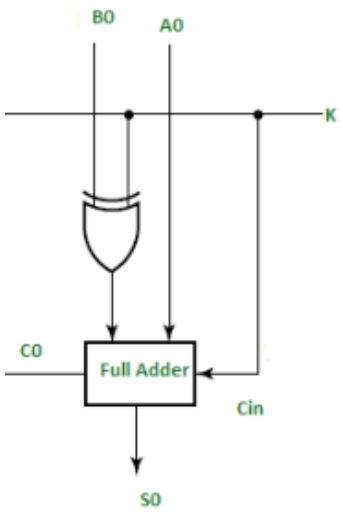

Figure 3.6: 1-bit adder/subtractor

VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
entity FA is
port (
A,B,Cin:in std_logic;
k:in std_logic;
Cout,S:out std_logic
);
end entity;
architecture adder of FA is signal B_xor : std_logic; begin
B_xor<=B xor k;
S <= A xor B_xor xor Cin; Cout <= (A and B_xor) or (A and Cin)
or (Cin and B_xor) ;
end architecture;
```

3.3.2 Schematic diagram

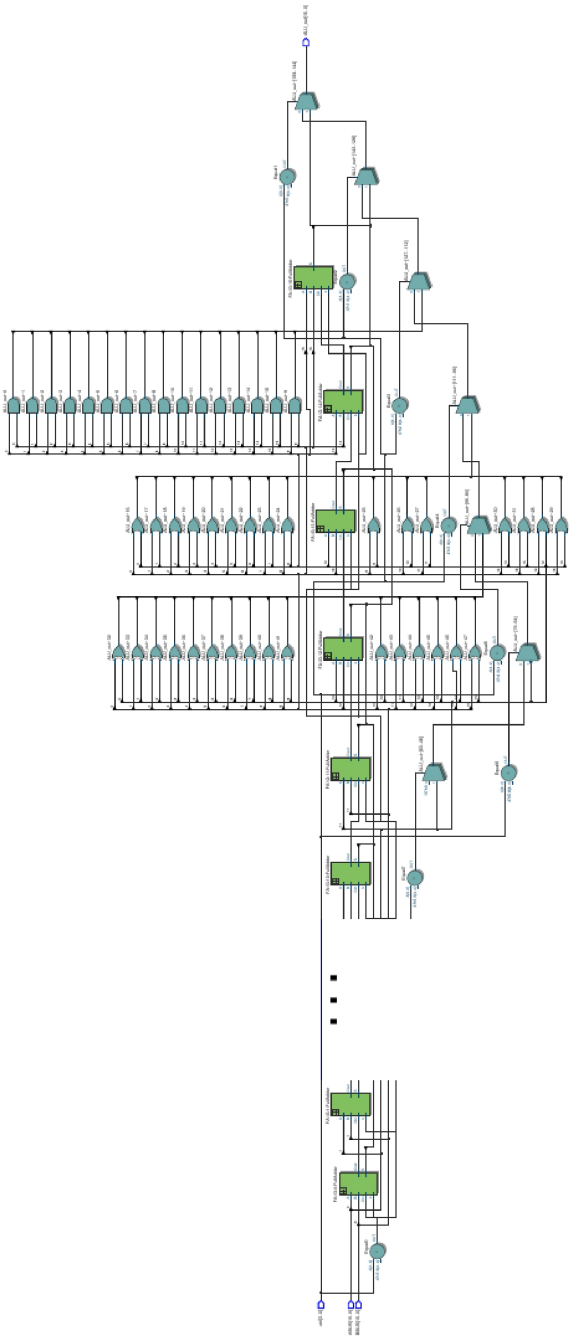

Figure 3.7: Schematic diagram of ALU

3.3.3 VHDL Code

VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;

entity ALU is
port (
ctrl:in std_logic_vector(3 downto 0);
ABUS,BBUS:in std_logic_vector(15 downto 0);
ALU_out:out std_logic_vector(15 downto 0)
);
end entity;

architecture bhv of ALU is
signal temp : std_logic_vector(15 downto 0);
signal Cout: std_logic_vector(16 downto 0);
signal exSignal: std_logic;
begin

exSignal<='1' when ctrl="0001" else '0';
Cout(0)<=exSignal;

G: for i in 0 to 15 generate
FullAdder : entity FA
port map (
A=>ABUS(i),
B=>BBUS(i),
Cin=>Cout(i),
Cout=>Cout(i+1),
k=>exSignal,
S=>temp(i));
end generate;
```

```
process (ctrl,ABUS,BBUS,temp)
begin
if ctrl="0000" then ALU_out<=temp; –add
elsif ctrl="0001" then ALU_out<=temp; –sub
elsif ctrl="0010" then ALU_out<=ABUS and BBUS;
elsif ctrl="0011" then ALU_out<=ABUS or BBUS;
elsif ctrl="0100" then ALU_out<=ABUS xor BBUS ;
elsif ctrl="0101" then ALU_out<=not ABUS ;
elsif ctrl="0110" then ALU_out<=ABUS ;
else ALU_out<=(others =>'0') ;
end if;
end process;
end architecture;
```

3.3.4 Simulation of ALU

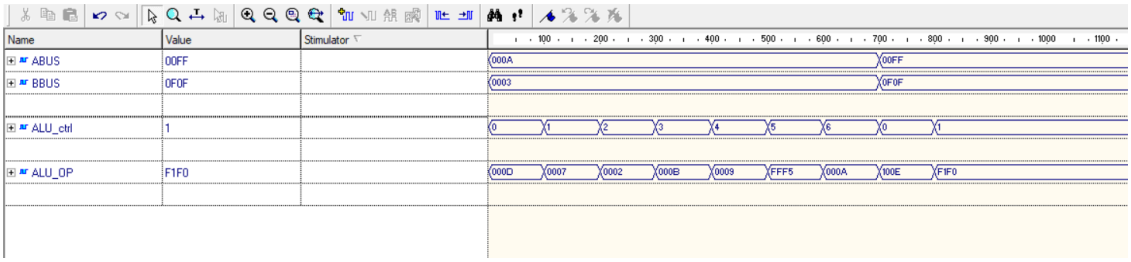

Figure 3.8: Simulation of ALU

3.4 Combinational Logic

3.4.1 description

The major modules of the combinational logic unit are ALU, Shifting Unit, and Nonlinear Lookup Operation

Structure

(i) **LUT :** The lower byte of `A_BUS` is fed directly into `non_linear_lookup`. The 8-bit LUT output is then concatenated with the unchanged upper byte of `A_BUS` to form the full 16-bit signal `temp1`.

(ii) **Shifter :** `B_BUS` and `CTRL` are forwarded to the `shifter`. Its output is stored in `temp3`.

(iii) **ALU :** Both `A_BUS` and `B_BUS` along with `CTRL` are forwarded to the `ALU`. Its result appears on `temp2`.

Output Multiplexer Logic

The MUX uses a two-stage decision based on `CTRL`:

- If `CTRL[3] = '0'`: select the ALU output
- If `CTRL[3] = '1'` and `CTRL[1:0] = "11"`: select the LUT output
- Otherwise (`CTRL[3] = '1'` and `CTRL[1:0] <> "11"`): select the Shifter output

3.4.2 Schematic diagram

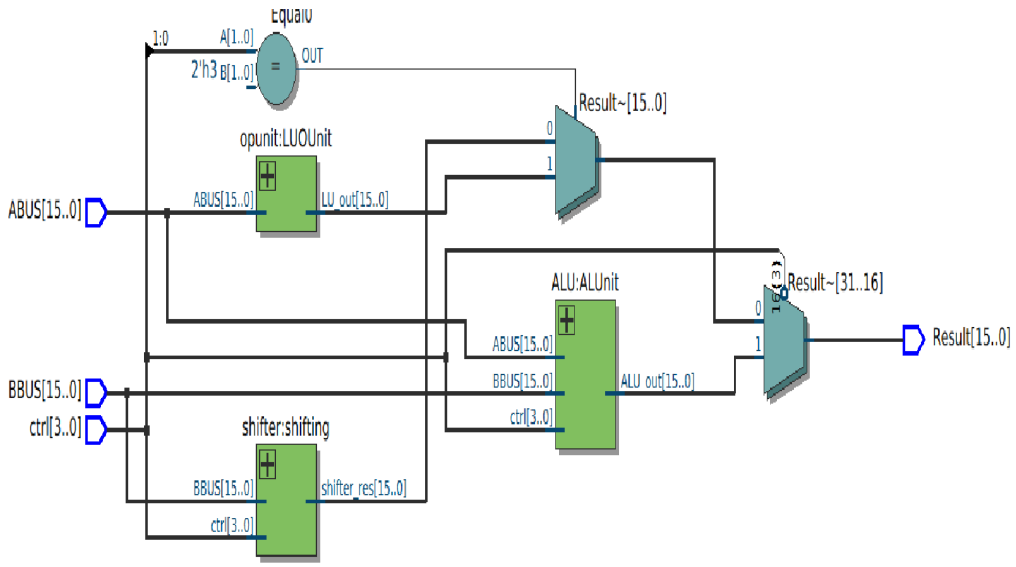

Figure 3.9: Schematic diagram of Combinational Logic

3.4.3 VHDL Code

VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
entity combinational is
port (
ctrl:in std_logic_vector(3 downto 0);
ABUS,BBUS:in std_logic_vector(15 downto 0);
Result:out std_logic_vector(15 downto 0)
);
end entity;

architecture com of combinational is
signal temp1,temp2,temp3: std_logic_vector(15 downto 0);
begin

LUOUnit: entity opunit
port map(
```

```
ABUS=>ABUS,
LU_out=>temp1
) ;

ALUnit: entity ALU port map (
ABUS=>ABUS,
BBUS=>BBUS,
ctrl=>ctrl,
ALU_out=>temp2 ) ;

shifting: entity shifter
port map(
BBUS=>BBUS,
ctrl=>ctrl,
shifter_res=>temp3
) ;

process (ctrl,temp1,temp2,temp3)
begin
if ctrl(3)='0' then – de bta3et el ALU
Result <= temp2;

elsif ctrl(1 downto 0)="11" then – de bta3et el LUT
Result <= temp1;

else
Result <= temp3; – de hatro7 le Shifter

end if;
end process;
end architecture;
```

3.4.4 Simulation of Combinational Logic Circuit

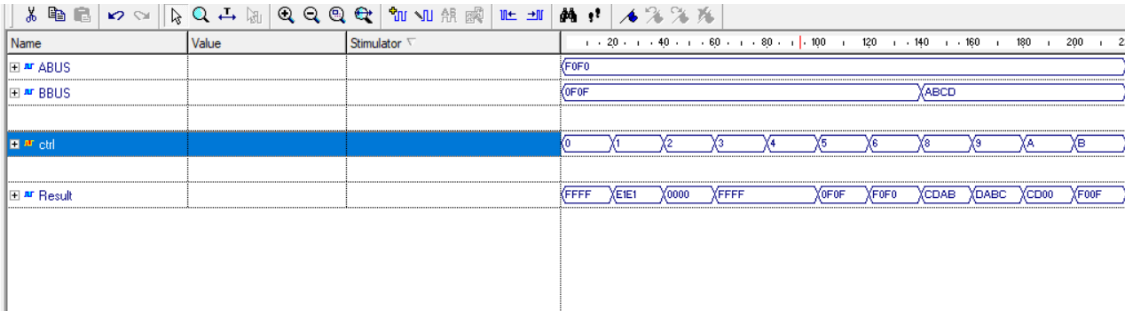

Figure 3.10: Schematic diagram of Combinational Logic

3.5 Register File

3.5.1 description

16 × 16-bit register file, which is a fundamental component used for temporary data storage and fast access in digital systems. The design contains 16 registers, each 16 bits wide, organized as an array and initialized to zero. It supports two simultaneous read operations and one write operation.

The read process is combinational, where the contents of the registers addressed by (ra) and (rb) are continuously available on the output buses (ABUS) and (BBUS).

The write process is synchronous and occurs on the rising edge of the clock signal. When the reset signal (rst) is activated, all registers are cleared to zero. Otherwise, if the enable signal (en) is set, the input data (res) is written into the register specified by the destination address rd.

This structure ensures efficient data access and controlled updates, making it suitable for use in processors and co-processor designs.

3.5.2 Schematic diagram

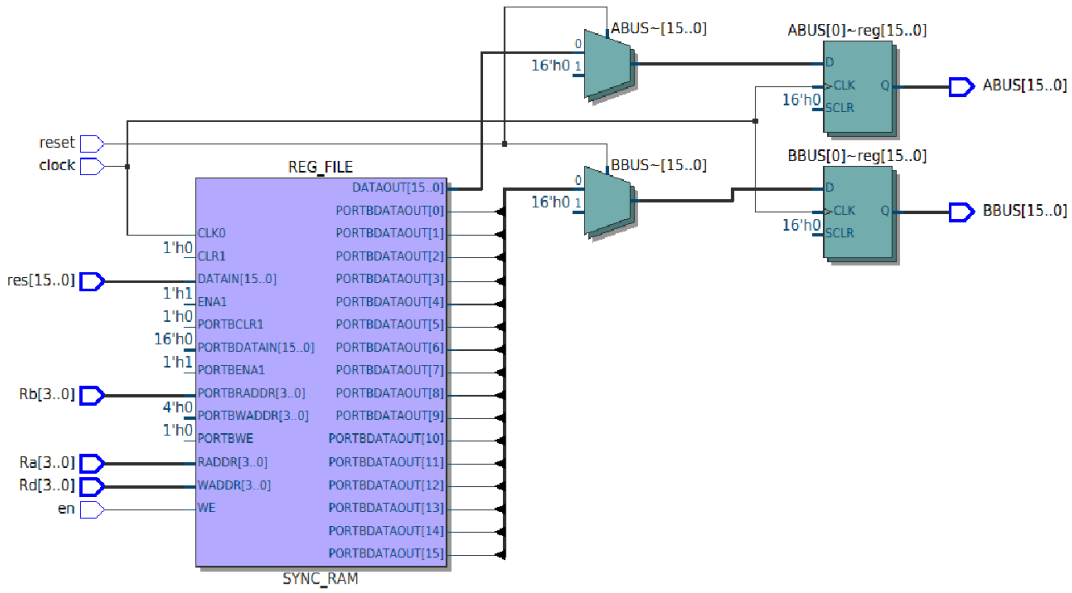

Figure 3.11: Schematic diagram of Register File

3.5.3 VHDL Code

VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity reg_file is
port(
clk, rst, en : in std_logic := '0';
ra, rb, rd : in std_logic_vector(3 downto 0);
res : in std_logic_vector(15 downto 0);
ABUS, BBUS : out std_logic_vector(15 downto 0)
);
end entity;

architecture behaviour of reg_file is
type reg_array is array (0 to 15) of std_logic_vector(15 downto 0);
```

```
signal REG : reg_array := (
0 => x"0024",1 => x"b456",
2 => x"3c07",3 => x"4b15",
4 => x"1354",5 => x"f407",
6 => x"daa5",7 => x"f123",
8 => x"ba54",9 => x"baa0",
10 => x"c902",11 => x"100b",
12 => x"c000",13 => x"a902",
14 => x"1564",15 => x"A030",
others => (others => '0') );
begin

-- READ (Asynchronous)
ABUS <= REG(to_integer(unsigned(ra)));
BBUS <= REG(to_integer(unsigned(rb)));

-- WRITE + RESET (Synchronous)
process(clk)
begin
if rising_edge(clk) then
    if rst = '1' then
      REG <= (others => (others => '0')); -- ? clears ALL registers
    elsif en = '1' then
      REG(to_integer(unsigned(rd))) <= res;
    end if;
end if;
end process;
end architecture;
```

3.5.4 Simulation of Register File

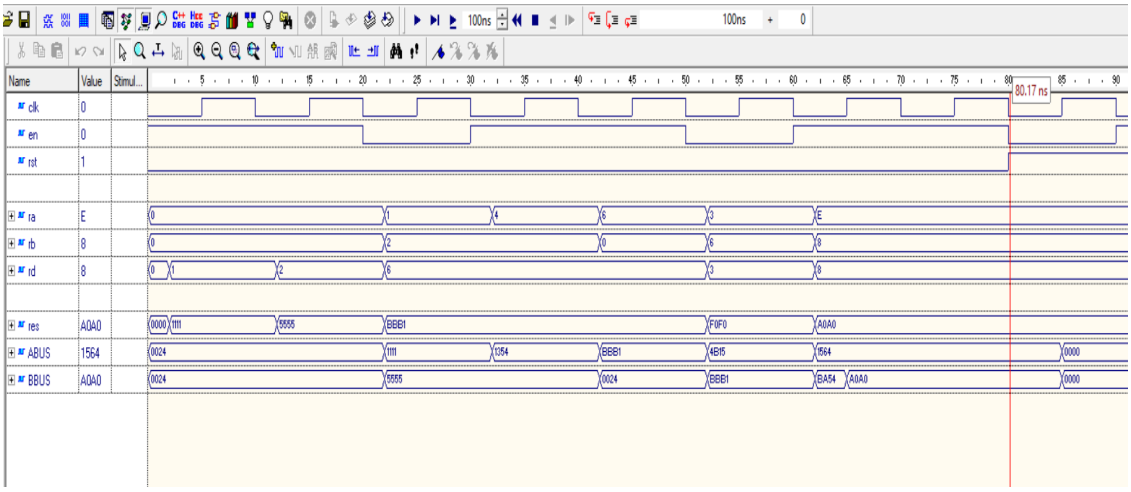

Figure 3.12: Simulation of Register File

3.6 High-Level Co-Processor Architecture

3.6.1 description

The co-processor is designed as an integrated system that combines a register file with a combinational logic unit to perform arithmetic and logical operations efficiently. The architecture enables data to be read from the register file, processed through the combinational unit, and optionally written back to the register file, forming a complete data processing cycle. This modular structure ensures clear separation between storage and computation while maintaining efficient data flow.

At the core of the design, the register file provides two output buses (`ABUS` and `BBUS`) which supply operands to the combinational logic block. The combinational unit performs operations based on the control signal and generates a result (`Result`), which is then fed back to the register file for storage. The entire system operates synchronously with the clock, ensuring stable and predictable behavior.

Key Features and Operation

- **Modular Integration**
  - The design consists of two main components:
    - Register File (data storage)
    - Combinational Logic Unit (data processing)
- **Data Flow Mechanism**
  - Operands are read from the register file via `ABUS` and `BBUS`.
  - These operands are processed by the combinational logic unit.
  - The computed result is written back to the register file input (`res`).
- **Synchronous Control**
  - All operations are synchronized with the rising edge of the clock.
  - Internal signals are updated in a sequential process to ensure timing stability.
- **Control Signal Handling**

  - The input control signal (`ctrl`) is stored in an internal register (`ctrl_result`).
  - The destination register (`Rd`) is buffered into (`Rd_tmp`) to avoid timing issues.

- **Reset Behavior**
  - When the reset signal (`rst`) is active:
    * Control signals are cleared.
    * Destination register is reset.
    * Write enable (`en`) is disabled.

- **Write Enable Logic**
  - The enable signal (`en`) controls writing back to the register file.
  - Writing is disabled when `ctrl = "0111"` (non-write operation).
  - For all other operations, writing is enabled.

Design Advantages

- Ensures stable operation through signal buffering.
- Provides efficient data processing using parallel read buses.
- Maintains clear separation between computation and storage.
- Supports flexible instruction control via the control signal.

3.6.2 Schematic diagram

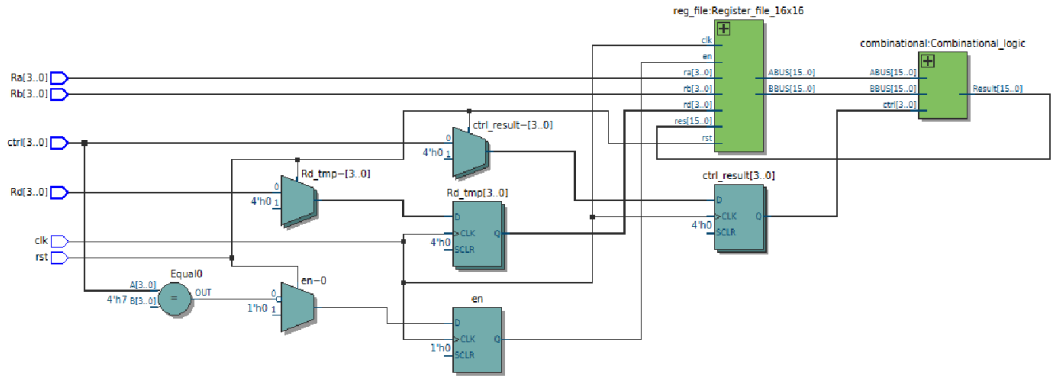

Figure 3.13: Schematic diagram of Co-Processor

3.6.3 VHDL Code

VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
entity CoProcessor is
port (
ctrl:in std_logic_vector(3 downto 0);
clk,rst:in std_logic;
Ra,Rb,Rd:in std_logic_vector(3 downto 0)
);
end entity ;

architecture rtl of CoProcessor is

signal ctrl_result: std_logic_vector(3 downto 0):=x"0";
signal ABUS,BBUS:std_logic_vector(15 downto 0);
signal Result: std_logic_vector(15 downto 0);
signal en :std_logic;
```

```
signal Rd_tmp: std_logic_vector(3 downto 0):=x"0";

begin

Combinational_logic:entity combinational
port map(
ctrl=>ctrl_result,
ABUS=>ABUS,BBUS=>BBUS,
Result=>Result
);

Register_file:entity reg_file
port map( clk=> clk, rst => rst,
en => en,
res => Result,
Ra => Ra,
Rb => Rb,
Rd => Rd_tmp,
ABUS => ABUS,
BBUS => BBUS);

process (clk)
begin

if(rising_edge(clk)) then
        if rst='1' then
            ctrl_result <=(others => '0');
            Rd_tmp <= (others => '0');
            en<='0';
        else
            Rd_tmp <= Rd;
            ctrl_result <= ctrl;
```

```
                if(ctrl = "0111") then – don't write to Reg_file
                    en <= '0';
                else
                    en <= '1';
                end if;
            end if ;
        end if;

        end process;
        end architecture;
```

3.6.4 Simulation of High level Co-Processor

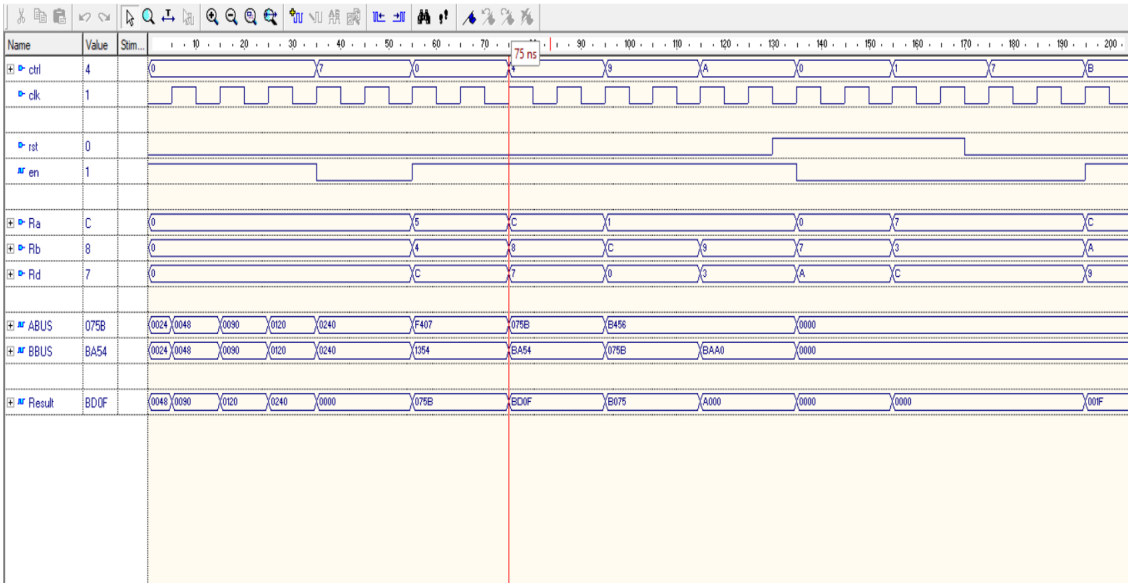

Figure 3.14: Simulation of Co-Processor

References

[1] fpga4student – Cryptographic Coprocessor Design in VHDL.

https://www.fpga4student.com/2017/07/cryptographic-coprocessor-design-in-vhdl.html

a popular educational website offering a wide range of VHDL and Verilog projects, tutorials, and tutorials for students and engineers. The site provides practical examples for FPGA development.

[2] Indonesian Journal of Engineering and Computer Science

https://share.google/md29uf2cPIc9KiWdo

The International Journal of Electrical and Computer Engineering Systems (IJEECS) is a peer-reviewed monthly journal by IAES publishing research on advances in electrical, electronics, and computer engineering, including telecommunications, IT, control systems, and power engineering.

[3] github/dana-66 – Cryptographic-coprocessor

https://github.com/dana-66/Cryptographic-coprocessor

An open-source reference implementation of a cryptographic coprocessor in VHDL. Consulted for architectural insights and component-level design patterns.

[4] github/aidansmyth95 – Crytpographic-Coprocessor

https://github.com/aidansmyth95/Crytpographic-Coprocessor

An open-source reference implementation of a cryptographic coprocessor in VHDL. Consulted for architectural insights and component-level design patterns.